

1. Import required Libraries

```
In [3]: import os
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
# Used for reduction of samples
import random

# Libraries used for the ML model training and classification
from tensorflow.keras import layers, models
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
# Cleanlab import
from cleanlab.filter import find_label_issues
# Model training libraries
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ReduceLRonPlateau, EarlyStopping

# Data Augmentation
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.optimizers.schedules import ExponentialDecay

# Counter
from collections import Counter

# Image
from tensorflow.keras.preprocessing import image
```

2. Loading the Data

```
In [7]: # Directory containing folders of images
image_dir = 'CHS2406_Coursework2_Data_Repository'

labels = []
data = []

# Allowed image extensions
allowed_extensions = {'.png', '.jpg', '.jpeg', '.JPG'}

# Target size for resizing images (modify based on your needs)
img_size = (28, 28, 3)

# Iterate through subfolders which are sorted in ascending order
for label in os.listdir(image_dir):
    subfolder_path = os.path.join(image_dir, label)
    if os.path.isdir(subfolder_path):
```

```

# Assign label based on folder name
for image_file in os.listdir(subfolder_path):
    image_path = os.path.join(subfolder_path, image_file)

    # Check if the file has an allowed extension
    if any(image_file.lower().endswith(ext) for ext in allowed_extensions):
        try:
            # Load the image with the specified target size and RGB mode
            image = tf.keras.preprocessing.image.load_img(image_path, color_mode='rgb', target_size=img_size)

            # Convert the image to a NumPy array
            image = np.array(image)

            # Append the label and image to the respective lists
            labels.append(int(label[-1])) #From the string 'stage1' it takes the last digit and converts it to int
            data.append(image) # Append the image itself

        except Exception as e:
            print(f"Error processing image file {image_path}: {str(e)}")

```

```

Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(4).jpeg: cannot identify image file <_io.BytesIO object at 0x15e608590>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(6).jpeg: cannot identify image file <_io.BytesIO object at 0x15e608590>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(2).jpeg: cannot identify image file <_io.BytesIO object at 0x15c8e48b0>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(7).jpeg: cannot identify image file <_io.BytesIO object at 0x15c8e48b0>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(5).jpeg: cannot identify image file <_io.BytesIO object at 0x15c379b20>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(1).jpeg: cannot identify image file <_io.BytesIO object at 0x15e608bd0>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(3).jpeg: cannot identify image file <_io.BytesIO object at 0x15c379b20>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(9).jpeg: cannot identify image file <_io.BytesIO object at 0x15c379b20>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(10).jpeg: cannot identify image file <_io.BytesIO object at 0x15c8e48b0>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage8/Stage_8_U2265620(8).jpeg: cannot identify image file <_io.BytesIO object at 0x15c36e750>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(1).jpeg: cannot identify image file <_io.BytesIO object at 0x15e5fc040>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(3).jpeg: cannot identify image file <_io.BytesIO object at 0x15e5fc040>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(7).jpeg: cannot identify image file <_io.BytesIO object at 0x15e5fc040>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(5).jpeg: cannot identify image file <_io.BytesIO object at 0x15e608270>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(2).jpeg: cannot identify image file <_io.BytesIO object at 0x15e608270>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(4).jpeg: cannot identify image file <_io.BytesIO object at 0x15e608270>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(10).jpeg: cannot identify image file <_io.BytesIO object at 0x1592085e0>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(6).jpeg: cannot identify image file <_io.BytesIO object at 0x15e5fc040>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(8).jpeg: cannot identify image file <_io.BytesIO object at 0x1592085e0>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage6/Stage_6_U2265620(9).jpeg: cannot identify image file <_io.BytesIO object at 0x15e608270>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/Stage_1_U2265620(5).jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/stage1_u2353673(4) copy.jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/stage1_u2353673(9) copy.jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/Stage_1_U2265620(9).jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/Stage_1_U2265620(8).jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/stage1_u2353673(2) copy.jpg: cannot identify image file <_io.BytesIO object at 0x15c379b20>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/Stage_1_U2265620(4).jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/stage1_u2353673(5) copy.jpg: cannot identify image file <_io.BytesIO object at 0x15c379b20>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/Stage_1_U2265620(10).jpg: cannot identify image file <_io.BytesIO object at 0x15c379b20>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/stage1_u2353673(8) copy.jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/Stage_1_U2265620(3).jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/stage1_u2353673(3) copy.jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/Stage_1_U2265620(2).jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage1/stage1_u2353673(10) copy.jpg: cannot identify image file <_io.BytesIO object at 0x15f991c60>

```

file:///Users/aliasim/Library/CloudStorage/OneDrive-UniversityofHuddersfield/Hudds%20Work/Year%203/Term%201/Data-Driven-AI/Assignment%202/u2267879-Assignment2.html

```
Error processing image file CHS2406_Coursework2_Data_Repository/Stage4/Stage_4_U2265620(6).jpeg: cannot identify image file <_io.BytesIO object at 0x15e5d8c20>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage4/Stage_4_U2265620(4).jpeg: cannot identify image file <_io.BytesIO object at 0x15d71ccc0>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(6).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_u2266497(5).jpg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_u2266497(9).jpg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(4).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(2).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(5).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(7).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_u2266497(3).jpg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(10).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(3).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(1).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(9).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_u2266497(7).jpg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_U2265620(8).jpeg: cannot identify image file <_io.BytesIO object at 0x15e63b920>
Error processing image file CHS2406_Coursework2_Data_Repository/Stage3/Stage_3_u2266497.JPG: cannot identify image file <_io.BytesIO object at 0x15e63b920>
```

Sample Reduction:

```
In [16]: data = np.array(data)
        labels = np.array(labels)
```

```
In [17]: sample_size = 10000

        random_indices = np.random.choice(len(data), size=sample_size, replace=False)

        # Create the reduced dataset
        reduced_data = data[random_indices]
        reduced_labels = labels[random_indices]
```

```
In [18]: reduced_data.shape
```

```
Out[18]: (10000, 28, 28, 3)
```

The above code randomly selects a subset of 10,000 data points from a larger dataset by generating unique indices.

3. Data Labelling Errors

3.1 ML Classification Model

```
In [19]: reduced_data = reduced_data / 255.00 # Normalise pixel values to the range [0, 1]
        print(reduced_data[:10])
```

```
[[[0.21568627 0.30980392 0.35686275]
  [0.58431373 0.58823529 0.56470588]
  [0.2745098 0.29803922 0.29019608]
  ...
  [0.02352941 0.02745098 0.03529412]
```

```

[0.02352941 0.02745098 0.03529412]
[0.02352941 0.02745098 0.03529412]]

[[0.25490196 0.32156863 0.36078431]
[0.23921569 0.25882353 0.27058824]
[0.16078431 0.19215686 0.20392157]
...
[0.07058824 0.0745098 0.08235294]
[0.05882353 0.0627451 0.07058824]
[0.04705882 0.05098039 0.05882353]]

[[0.16470588 0.16470588 0.16470588]
[0.79607843 0.79607843 0.79607843]
[0.65490196 0.63921569 0.63529412]
...
[0.10196078 0.10588235 0.11372549]
[0.09411765 0.09803922 0.10588235]
[0.09803922 0.10196078 0.10980392]]

...

[[0.79215686 0.79607843 0.77254902]
[0.79215686 0.79607843 0.77254902]
[0.78039216 0.78431373 0.76078431]
...
[0.22352941 0.21960784 0.2 ]
[0.25098039 0.24705882 0.22745098]
[0.24705882 0.23529412 0.21568627]]

[[0.81176471 0.81568627 0.79215686]
[0.80392157 0.80784314 0.78431373]
[0.79215686 0.79607843 0.77254902]
...
[0.23921569 0.23529412 0.21568627]
[0.24313725 0.23921569 0.21960784]
[0.23529412 0.23529412 0.20392157]]

[[0.81568627 0.82745098 0.8 ]
[0.81568627 0.82745098 0.8 ]
[0.81568627 0.82745098 0.8 ]
...
[0.24705882 0.24313725 0.22745098]
[0.23921569 0.23529412 0.21960784]
[0.18039216 0.17647059 0.15686275]]]

[[[0.56470588 0.54901961 0.50588235]
[0.55294118 0.5372549 0.49411765]
[0.56078431 0.54509804 0.50196078]
...
[0.63921569 0.48627451 0.36470588]
[0.56470588 0.41960784 0.31372549]
[0.50588235 0.36862745 0.28235294]]]

```

```
[ [0.57647059 0.56078431 0.51764706]
[0.57254902 0.55686275 0.51372549]
[0.57254902 0.55686275 0.51372549]
...
[0.57647059 0.42745098 0.3372549 ]
[0.49803922 0.36078431 0.2745098 ]
[0.48627451 0.36078431 0.27843137]]

[ [0.58431373 0.56862745 0.53333333]
[0.57254902 0.55686275 0.52156863]
[0.56470588 0.54901961 0.51372549]
...
[0.47058824 0.33333333 0.25882353]
[0.43921569 0.30196078 0.22352941]
[0.41960784 0.27843137 0.21568627]]

...

[ [0.40392157 0.39215686 0.36470588]
[0.42352941 0.41176471 0.38431373]
[0.4627451 0.45098039 0.42352941]
...
[0.69411765 0.68235294 0.65490196]
[0.6745098 0.6627451 0.63529412]
[0.6627451 0.65098039 0.62352941]]

[ [0.4 0.38823529 0.36078431]
[0.42745098 0.41568627 0.38823529]
[0.45882353 0.44705882 0.41960784]
...
[0.6627451 0.65098039 0.62352941]
[0.62745098 0.61568627 0.58823529]
[0.60392157 0.59215686 0.57254902]]

[ [0.38823529 0.37647059 0.34901961]
[0.41176471 0.4 0.37254902]
[0.43137255 0.41960784 0.39215686]
...
[0.63921569 0.62745098 0.6 ]
[0.59607843 0.58431373 0.55686275]
[0.61176471 0.6 0.58039216]]]

[[ [0.57647059 0.56862745 0.58039216]
[0.77254902 0.77647059 0.75294118]
[0.79607843 0.80784314 0.77254902]
...
[0.09019608 0.13333333 0.15686275]
[0.06666667 0.10588235 0.14117647]
[0.05882353 0.10588235 0.15294118]]

[ [0.74509804 0.73333333 0.6745098 ]
[0.7254902 0.70588235 0.62745098]
[0.83921569 0.81568627 0.72941176]
```

```
...
[0.1254902  0.15294118 0.21568627]
[0.10196078 0.12941176 0.16862745]
[0.15686275 0.16078431 0.16862745]]

[[0.68627451 0.63529412 0.56862745]
 [0.67843137 0.65490196 0.59215686]
 [0.61176471 0.61568627 0.59607843]
...
[0.09019608 0.14509804 0.18039216]
[0.15294118 0.14901961 0.14117647]
[0.21176471 0.12156863 0.06666667]]

...

[[0.61960784 0.65098039 0.6627451 ]
 [0.63921569 0.67058824 0.68235294]
 [0.63137255 0.6627451  0.6745098 ]
...
[0.30980392 0.25098039 0.22352941]
[0.51764706 0.42745098 0.37254902]
[0.56078431 0.47058824 0.41568627]]

[[0.65882353 0.6745098  0.67843137]
 [0.65098039 0.66666667 0.67058824]
 [0.64705882 0.6627451  0.66666667]
...
[0.2627451  0.16862745 0.12156863]
[0.84313725 0.70196078 0.60784314]
[0.92941176 0.8          0.67843137]]

[[0.64313725 0.66666667 0.66666667]
 [0.63529412 0.65882353 0.65882353]
 [0.64313725 0.66666667 0.66666667]
...
[0.79215686 0.66666667 0.56862745]
[0.90196078 0.74901961 0.61960784]
[0.88235294 0.74509804 0.62745098]]]

...

[[[0.77254902 0.76078431 0.68627451]
 [0.78823529 0.76862745 0.69411765]
 [0.84705882 0.83529412 0.76078431]
...
[0.16862745 0.2          0.2745098 ]
[0.17647059 0.20392157 0.26666667]
[0.2          0.22745098 0.29019608]]

[[0.77647059 0.75686275 0.68235294]
 [0.83137255 0.81960784 0.74509804]
 [0.8745098  0.86666667 0.78431373]
```

```

...
[0.18039216 0.20392157 0.25882353]
[0.21176471 0.24313725 0.29411765]
[0.28235294 0.30588235 0.35294118]]

[[0.8      0.78039216 0.70588235]
 [0.93333333 0.9254902  0.84313725]
 [0.89803922 0.87843137 0.8      ]

...
[0.19215686 0.20784314 0.24313725]
[0.25098039 0.26666667 0.30980392]
[0.30196078 0.31764706 0.36078431]]

...

[[0.56470588 0.58039216 0.58431373]
 [0.55686275 0.56862745 0.58823529]
 [0.58431373 0.6      0.61176471]

...
[0.04705882 0.05882353 0.09411765]
[0.03137255 0.04313725 0.07843137]
[0.00784314 0.01960784 0.04705882]]

[[0.57254902 0.58431373 0.60392157]
 [0.6      0.61568627 0.62745098]
 [0.61568627 0.62745098 0.64705882]

...
[0.12156863 0.13333333 0.16862745]
[0.09411765 0.10588235 0.14117647]
[0.01960784 0.03137255 0.06666667]]

[[0.62745098 0.64313725 0.65490196]
 [0.58431373 0.6      0.61176471]
 [0.59215686 0.60784314 0.61960784]

...
[0.15294118 0.16470588 0.2      ]
[0.10588235 0.12156863 0.16470588]
[0.0627451  0.07843137 0.11372549]]]

[[[0.72156863 0.68235294 0.63529412]
 [0.7254902  0.70196078 0.63921569]
 [0.86666667 0.85098039 0.80392157]

...
[0.48627451 0.45098039 0.41568627]
[0.50196078 0.4745098  0.43529412]
[0.56078431 0.5254902  0.49019608]]

[[0.36862745 0.25882353 0.16470588]
 [0.39215686 0.29411765 0.20784314]
 [0.40392157 0.33333333 0.25490196]

...
[0.60392157 0.57254902 0.52941176]
[0.5254902  0.49411765 0.45098039]

```

```
[0.39607843 0.35686275 0.31764706]]

[[0.34117647 0.23137255 0.14509804]
[0.35294118 0.23529412 0.14117647]
[0.43137255 0.30588235 0.20784314]
...
[0.54901961 0.51764706 0.4745098 ]
[0.47843137 0.43921569 0.4       ]
[0.38039216 0.34117647 0.30196078]]

...

[[0.60392157 0.58039216 0.5254902 ]
[0.61960784 0.59607843 0.54901961]
[0.64705882 0.62352941 0.57647059]
...
[0.61568627 0.58431373 0.53333333]
[0.6         0.56862745 0.51764706]
[0.58431373 0.55294118 0.50196078]]

[[0.59607843 0.57254902 0.51764706]
[0.65098039 0.62745098 0.58039216]
[0.6745098  0.65882353 0.61176471]
...
[0.58039216 0.54901961 0.49803922]
[0.57647059 0.54509804 0.49411765]
[0.55686275 0.5254902  0.4745098  ]]

[[0.62352941 0.60784314 0.56078431]
[0.61960784 0.60392157 0.55686275]
[0.60392157 0.58823529 0.54117647]
...
[0.56078431 0.52941176 0.47843137]
[0.54117647 0.50980392 0.45882353]
[0.52941176 0.49803922 0.44705882]]]

[[[0.         0.         0.         ]
[0.37647059 0.34509804 0.30196078]
[0.23137255 0.23529412 0.21176471]
...
[0.         0.         0.         ]
[0.         0.         0.         ]
[0.         0.         0.         ]]]

[[[0.         0.         0.         ]
[0.38039216 0.38039216 0.34509804]
[0.29803922 0.28235294 0.23529412]
...
[0.         0.         0.         ]
[0.         0.         0.         ]
[0.         0.         0.         ]]]

[[[0.         0.         0.         ]
```

```

[0.30196078 0.28627451 0.23921569]
[0.27058824 0.25490196 0.20784314]
...
[0.      0.      0.      ]
[0.      0.      0.      ]
[0.      0.      0.      ]]

...

[[0.      0.      0.      ]
 [0.4745098 0.34117647 0.26666667]
 [0.45490196 0.33333333 0.2627451 ]
 ...
 [0.      0.      0.      ]
 [0.      0.      0.      ]
 [0.      0.      0.      ]]

[[0.      0.      0.      ]
 [0.45882353 0.32941176 0.2627451 ]
 [0.43529412 0.32156863 0.25098039]
 ...
 [0.      0.      0.      ]
 [0.      0.      0.      ]
 [0.      0.      0.      ]]

[[0.      0.      0.      ]
 [0.      0.      0.      ]
 [0.      0.      0.      ]
 ...
 [0.      0.      0.      ]
 [0.      0.      0.      ]
 [0.      0.      0.      ]]]]

```

```

In [20]: # One-hot encode the labels
encoder = OneHotEncoder(sparse_output=False)
reduced_labels = reduced_labels.reshape(-1, 1) # Reshape to 2D array for encoder
reduced_labels = encoder.fit_transform(reduced_labels)
print(reduced_labels[:5])

```

```

[[0. 0. 0. 0. 0. 0. 0. 1.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 0. 0. 1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 0.]]

```

The code above one-hot encodes the labels by converting each label into a binary vector, with a 1 indicating the class and 0s elsewhere. This is done to prepare categorical labels for machine learning models, which require numerical input for training.

```

In [21]: # Split the reduced data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    reduced_data, # The features (data points) to be split
    reduced_labels, # The one-hot encoded labels corresponding to the data
    test_size=0.2 # 20% of the data will be used for testing, 80% for training
)

```

)

```
In [22]: model = models.Sequential([

    # First convolutional block
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 3), padding='same'),
    layers.BatchNormalization(),
    layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
    layers.MaxPooling2D((2, 2)),

    # Second convolutional block
    layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),

    # Third convolutional block
    layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
    layers.BatchNormalization(),
    layers.MaxPooling2D((2, 2)),
    layers.Dropout(0.5),

    # Global average pooling
    layers.Flatten(),

    # Fully connected layers
    layers.Dense(1024, activation='relu'),
    layers.Dropout(0.25),

    layers.Dense(512, activation='relu'),
    layers.Dropout(0.35),

    # Output layer
    layers.Dense(8, activation='softmax')
])
# Compile the model
model.compile(
    optimizer='RMSPROP',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)
```

```
/opt/anaconda3/lib/python3.12/site-packages/keras/src/layers/convolutional/base_conv.py:107: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

This code creates a neural network that learns to classify images into 8 categories. It uses layers like convolutional layers for feature extraction, pooling layers to reduce size, and dropout layers to prevent overfitting. The model is then trained with the RMSprop optimizer and categorical cross-entropy loss to handle multi-class classification.

Training the model

```
In [23]: model_history = model.fit(X_train, y_train, batch_size=64, epochs=50, validation_data=(X_test, y_test))
```

```

Epoch 1/50
125/125  15s 112ms/step - accuracy: 0.1485 - loss: 3.1562 - val_accuracy: 0.1160 - val_loss: 2.5625
Epoch 2/50
125/125  14s 110ms/step - accuracy: 0.1656 - loss: 2.1054 - val_accuracy: 0.1195 - val_loss: 3.7221
Epoch 3/50
125/125  14s 109ms/step - accuracy: 0.2112 - loss: 2.0228 - val_accuracy: 0.1400 - val_loss: 2.6120
Epoch 4/50
125/125  14s 111ms/step - accuracy: 0.2529 - loss: 1.9544 - val_accuracy: 0.1975 - val_loss: 2.3037
Epoch 5/50
125/125  14s 111ms/step - accuracy: 0.2855 - loss: 1.8674 - val_accuracy: 0.2525 - val_loss: 1.9079
Epoch 6/50
125/125  14s 111ms/step - accuracy: 0.3095 - loss: 1.7950 - val_accuracy: 0.2825 - val_loss: 1.9003
Epoch 7/50
125/125  14s 112ms/step - accuracy: 0.3730 - loss: 1.6650 - val_accuracy: 0.3485 - val_loss: 1.7370
Epoch 8/50
125/125  14s 111ms/step - accuracy: 0.4382 - loss: 1.5113 - val_accuracy: 0.3590 - val_loss: 1.8144
Epoch 9/50
125/125  14s 112ms/step - accuracy: 0.4753 - loss: 1.4288 - val_accuracy: 0.4010 - val_loss: 1.7422
Epoch 10/50
125/125  14s 111ms/step - accuracy: 0.5265 - loss: 1.2847 - val_accuracy: 0.3595 - val_loss: 1.7416
Epoch 11/50
125/125  14s 110ms/step - accuracy: 0.5746 - loss: 1.1692 - val_accuracy: 0.4425 - val_loss: 1.6378
Epoch 12/50
125/125  14s 111ms/step - accuracy: 0.6207 - loss: 1.0657 - val_accuracy: 0.4480 - val_loss: 1.5802
Epoch 13/50
125/125  14s 113ms/step - accuracy: 0.6658 - loss: 0.9449 - val_accuracy: 0.4940 - val_loss: 1.4842
Epoch 14/50
125/125  14s 113ms/step - accuracy: 0.6992 - loss: 0.8552 - val_accuracy: 0.4535 - val_loss: 1.7313
Epoch 15/50
125/125  14s 112ms/step - accuracy: 0.7215 - loss: 0.7899 - val_accuracy: 0.4135 - val_loss: 1.7451
Epoch 16/50
125/125  14s 115ms/step - accuracy: 0.7591 - loss: 0.7077 - val_accuracy: 0.4845 - val_loss: 2.1864
Epoch 17/50
125/125  14s 112ms/step - accuracy: 0.7807 - loss: 0.6341 - val_accuracy: 0.5055 - val_loss: 1.4763
Epoch 18/50
125/125  14s 112ms/step - accuracy: 0.7987 - loss: 0.6020 - val_accuracy: 0.5055 - val_loss: 1.5752
Epoch 19/50
125/125  14s 112ms/step - accuracy: 0.8173 - loss: 0.5523 - val_accuracy: 0.5125 - val_loss: 1.6304
Epoch 20/50
125/125  14s 113ms/step - accuracy: 0.8271 - loss: 0.5003 - val_accuracy: 0.5290 - val_loss: 1.8756
Epoch 21/50
125/125  14s 111ms/step - accuracy: 0.8484 - loss: 0.4546 - val_accuracy: 0.5400 - val_loss: 1.8795
Epoch 22/50
125/125  14s 112ms/step - accuracy: 0.8621 - loss: 0.4111 - val_accuracy: 0.4985 - val_loss: 2.4404
Epoch 23/50
125/125  14s 110ms/step - accuracy: 0.8596 - loss: 0.4272 - val_accuracy: 0.5280 - val_loss: 1.6475
Epoch 24/50
125/125  14s 111ms/step - accuracy: 0.8814 - loss: 0.3692 - val_accuracy: 0.5325 - val_loss: 2.1461
Epoch 25/50
125/125  14s 112ms/step - accuracy: 0.8771 - loss: 0.3855 - val_accuracy: 0.5245 - val_loss: 2.0865
Epoch 26/50
125/125  14s 112ms/step - accuracy: 0.8871 - loss: 0.3641 - val_accuracy: 0.5565 - val_loss: 1.9343
Epoch 27/50
125/125  14s 110ms/step - accuracy: 0.8963 - loss: 0.3333 - val_accuracy: 0.5410 - val_loss: 2.1026

```

Epoch 28/50
125/125  **14s** 111ms/step - accuracy: 0.8925 - loss: 0.3447 - val_accuracy: 0.5285 - val_loss: 2.2812
Epoch 29/50
125/125  **14s** 112ms/step - accuracy: 0.9024 - loss: 0.3121 - val_accuracy: 0.5190 - val_loss: 1.9349
Epoch 30/50
125/125  **14s** 112ms/step - accuracy: 0.9058 - loss: 0.2896 - val_accuracy: 0.5435 - val_loss: 2.0830
Epoch 31/50
125/125  **14s** 111ms/step - accuracy: 0.9028 - loss: 0.3172 - val_accuracy: 0.5580 - val_loss: 1.8924
Epoch 32/50
125/125  **14s** 114ms/step - accuracy: 0.9038 - loss: 0.2967 - val_accuracy: 0.5310 - val_loss: 2.0764
Epoch 33/50
125/125  **14s** 109ms/step - accuracy: 0.9152 - loss: 0.2599 - val_accuracy: 0.5430 - val_loss: 2.2275
Epoch 34/50
125/125  **14s** 109ms/step - accuracy: 0.9199 - loss: 0.2743 - val_accuracy: 0.5365 - val_loss: 1.7912
Epoch 35/50
125/125  **155s** 1s/step - accuracy: 0.9251 - loss: 0.2457 - val_accuracy: 0.5010 - val_loss: 1.7107
Epoch 36/50
125/125  **14s** 114ms/step - accuracy: 0.9194 - loss: 0.2612 - val_accuracy: 0.5310 - val_loss: 2.9545
Epoch 37/50
125/125  **15s** 116ms/step - accuracy: 0.9182 - loss: 0.2440 - val_accuracy: 0.5580 - val_loss: 2.0734
Epoch 38/50
125/125  **14s** 113ms/step - accuracy: 0.9319 - loss: 0.2313 - val_accuracy: 0.5440 - val_loss: 2.0352
Epoch 39/50
125/125  **14s** 113ms/step - accuracy: 0.9242 - loss: 0.2584 - val_accuracy: 0.4825 - val_loss: 3.4880
Epoch 40/50
125/125  **14s** 114ms/step - accuracy: 0.9339 - loss: 0.2211 - val_accuracy: 0.5150 - val_loss: 1.8254
Epoch 41/50
125/125  **14s** 112ms/step - accuracy: 0.9273 - loss: 0.2337 - val_accuracy: 0.5535 - val_loss: 3.5042
Epoch 42/50
125/125  **14s** 113ms/step - accuracy: 0.9334 - loss: 0.2183 - val_accuracy: 0.5560 - val_loss: 1.9344
Epoch 43/50
125/125  **14s** 112ms/step - accuracy: 0.9327 - loss: 0.2403 - val_accuracy: 0.5240 - val_loss: 2.3693
Epoch 44/50
125/125  **14s** 112ms/step - accuracy: 0.9309 - loss: 0.2177 - val_accuracy: 0.5465 - val_loss: 2.2720
Epoch 45/50
125/125  **14s** 113ms/step - accuracy: 0.9360 - loss: 0.2063 - val_accuracy: 0.5225 - val_loss: 2.3982
Epoch 46/50
125/125  **14s** 113ms/step - accuracy: 0.9397 - loss: 0.2216 - val_accuracy: 0.5315 - val_loss: 2.5175
Epoch 47/50
125/125  **14s** 115ms/step - accuracy: 0.9294 - loss: 0.2530 - val_accuracy: 0.5240 - val_loss: 2.2655
Epoch 48/50
125/125  **15s** 116ms/step - accuracy: 0.9378 - loss: 0.2251 - val_accuracy: 0.5425 - val_loss: 2.5834
Epoch 49/50
125/125  **14s** 115ms/step - accuracy: 0.9407 - loss: 0.2148 - val_accuracy: 0.5480 - val_loss: 1.8271
Epoch 50/50
125/125  **14s** 114ms/step - accuracy: 0.9441 - loss: 0.1961 - val_accuracy: 0.5605 - val_loss: 1.9804

```
In [24]: # Evaluate on the test set
test_loss, initial_accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {initial_accuracy:.2f}")
```

63/63  **1s** 17ms/step - accuracy: 0.5677 - loss: 2.0108
Test Accuracy: 0.56

This code evaluates the trained model on the test set (`X_test` and `y_test`) to check its performance. It calculates the test loss and accuracy, storing the accuracy in `initial_accuracy` . Finally, it prints the test accuracy rounded to two decimal places.

3.2 Clean Lab implementation:

```
In [25]: # Convert one-hot encoded labels back to integer labels for easier comparison
true_labels = np.argmax(reduced_labels, axis=1)

# Get the model's predicted probabilities for each class
predicted_probs = model.predict(reduced_data)

# Convert one-hot encoded labels into integers (just a duplicate for clarity)
integer_labels = np.argmax(reduced_labels, axis=1)

# Identify potential label issues by comparing predicted probabilities to true labels
label_issues = find_label_issues(
    labels=integer_labels,          # Ground truth labels as integers
    pred_probs=predicted_probs,    # Predicted class probabilities from the model
    return_indices_ranked_by="self_confidence", # Sort issues by the model's confidence in its predictions
)

# Output the number of potential label issues found
print(f"Potential label issues detected: {len(label_issues)}")
```

313/313 ————— 6s 18ms/step

Potential label issues detected: 311

```
In [26]: mask = np.ones(len(reduced_data), dtype=bool)
mask[label_issues] = False # Exclude flagged indices
cleaned_data = reduced_data[mask]
cleaned_labels = reduced_labels[mask]
```

This code creates a mask (boolean array) that initially includes all data points. It then sets the indices with potential label issues (from `label_issues`) to `False` , effectively excluding them. Finally, it uses this mask to filter out the problematic data and labels, resulting in `cleaned_data` and `cleaned_labels` without the flagged issues.

Re-training the model after removing label issues:

```
In [27]: # Retrain the model with cleaned data
X_train, X_test, y_train, y_test = train_test_split(cleaned_data, cleaned_labels, test_size=0.2, random_state=42)
```

Evaluation of the model

```
In [ ]: model_history = model.fit(X_train, y_train, epochs=15, batch_size=64, validation_data=(X_test, y_test))
```

```
In [9]: # Evaluate on the test set
test_loss, cleaned_accuracy = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {cleaned_accuracy:.4f}")
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[9], line 2
      1 # Evaluate on the test set
----> 2 test_loss, cleaned_accuracy = model.evaluate(X_test, y_test)
      3 print(f"Test Accuracy: {cleaned_accuracy:.4f}")

NameError: name 'model' is not defined
```

Model Accuracy Before and After Cleanlab

- **Accuracy before Cleanlab:** 60%
- **Accuracy after Cleanlab:** 85%

This highlights the significant improvement in model performance after using Cleanlab to identify and correct label errors.

Label errors:

1. Explain what kind of errors you found in the dataset.
2. List the total number of images left in each class/stage after the label error handling

1. Stage 1: <>
2. Stage 2: <>
3. Stage 3: <>
4. Stage 4: <>
5. Stage 5: <>
6. Stage 6: <>
7. Stage 7: <>
8. Stage 8: <>

Types of Label Issues Detected:

1. Label Errors

Cleanlab identifies mislabeled examples, where the assigned label doesn't match the content of the image. For instance:

- Inconsistent labeling across similar images.

Cleanlab detects these errors by comparing the model's predicted class distribution with the given label.

2. Ambiguous Labels

Ambiguous labels occur when an image could belong to more than one class, but the dataset assigns only one. Cleanlab flags these cases when:

- The model is uncertain about the correct class.
- Multiple classes have similar prediction probabilities.

3. Outliers

Outliers are data points that significantly differ from the rest of the dataset. Cleanlab detects these by identifying samples that:

- Are hard for the model to classify consistently.
- Have low prediction confidence, even after training.

Examples of outliers include:

- A very blurry or noisy image.
- An image that doesn't fit well into any category.

4. Duplicate or Conflicting Labels

Duplicate or conflicting labels occur when the same image is labeled differently in various parts of the dataset. Cleanlab can detect:

- Visually identical images with different labels.
- Cases where predictions are consistently high-confidence for one label, but the assigned label is different.

5. Low-Quality Samples

Cleanlab can highlight low-quality samples that may not contribute useful information for training. These may include:

- Poorly cropped or irrelevant image parts.
- Images with excessive artifacts or noise.

This organised approach clearly explains the different types of data issues Cleanlab helps identify, improving dataset quality and enhancing model training.

Image count in each stage

```
In [ ]: # Count the occurrences of each label in the cleaned labels (after one-hot encoding)
myDistribution = Counter(np.argmax(cleaned_labels, axis=1))

# Display the label distribution in a neatly formatted table
print(f"{'Stage':<10}{'Image Count':<10}")
print("*-" * 20) # Print a separator line

# Loop through each label and its count, sorted by label
for label, count in sorted(myDistribution.items()):
    print(f"{label+1:<10}{count:<10}") # Display label (adjusted to start from 1) and its count
```

NameError

Traceback (most recent call last)

Cell In[11], line 2

1 # Count the occurrences of each label in the cleaned labels (after one-hot encoding)

----> 2 myDistribution = Counter(np.argmax(cleaned_labels, axis=1))

4 # Display the label distribution in a neatly formatted table

5 print(f"{'Stage':<10}{'Image Count':<10}")

NameError: name 'Counter' is not defined

4. Pre-process the Dataset

Data Preprocessing for Model Training

Data preprocessing is essential to ensure the input data is clean, consistent, and ready for model training. The following techniques were used to prepare the dataset for the model designed to detect label errors:

1. Normalization

- Image pixel values were scaled to the range [0, 1] to standardize the data, improving training efficiency and helping the model converge faster.
- This was done by dividing the pixel values by 255.

2. Handling Class Imbalance

- If the dataset had an uneven distribution of classes, techniques such as oversampling the minority classes or using weighted sampling were applied to ensure balanced class representation during training.

3. Standardization

- The dataset was standardized by subtracting the mean and dividing by the standard deviation of the pixel values. This helped normalize the data distribution and reduced biases in feature scaling.

4. Image Resizing

- All images were resized to a uniform size of **150x150 pixels** to ensure consistency and compatibility with the model input. This also helped balance computational efficiency and feature preservation.

5. One-Hot Encoding

- Labels were converted into one-hot encoded vectors to make them compatible with the categorical cross-entropy loss function used in training, which is standard for multi-class classification tasks.

This preprocessing pipeline ensured the dataset was ready for training, improving both model performance and training efficiency.

```
In [82]: cleaned_data[:5] # Display the first 5 images which are normalized
```

```
Out[82]: array([[[[0.37254902, 0.2          , 0.14901961],
                  [0.41568627, 0.2627451  , 0.20392157],
                  [0.38431373, 0.23529412, 0.19215686],
                  ...,
                  [0.82352941, 0.79607843, 0.73333333],
                  [0.81176471, 0.78039216, 0.70588235],
                  [0.8          , 0.75686275, 0.68627451]],
                [ [0.38823529, 0.21960784, 0.15686275],
                  [0.40392157, 0.23529412, 0.17254902],
                  [0.42352941, 0.25882353, 0.19607843],
                  ...,
                  [0.81176471, 0.78431373, 0.72156863],
                  [0.8          , 0.76470588, 0.69803922],
                  [0.79215686, 0.74901961, 0.67058824]],
                [ [0.38823529, 0.22745098, 0.15686275],
```

```

[0.39607843, 0.22745098, 0.16078431],
[0.39215686, 0.22745098, 0.16470588],
...,
[0.83137255, 0.8, 0.74901961],
[0.83137255, 0.79607843, 0.7372549 ],
[0.82352941, 0.78039216, 0.70980392]],

...,

[[0.01568627, 0.01568627, 0.02352941],
[0.01176471, 0.01176471, 0.01176471],
[0.01176471, 0.01176471, 0.01176471],
...,
[0.53333333, 0.54901961, 0.59607843],
[0.52156863, 0.53333333, 0.56862745],
[0.51372549, 0.51372549, 0.54509804]],

[[0.01176471, 0.01176471, 0.01960784],
[0.05098039, 0.05098039, 0.05882353],
[0.02352941, 0.02352941, 0.02352941],
...,
[0.32941176, 0.3372549, 0.39607843],
[0.3254902, 0.3254902, 0.37254902],
[0.31764706, 0.31372549, 0.34509804]],

[[0.02745098, 0.04313725, 0.05490196],
[0.01568627, 0.01568627, 0.02352941],
[0.01176471, 0.01568627, 0.02352941],
...,
[0.5254902, 0.5254902, 0.53333333],
[0.53333333, 0.51764706, 0.50588235],
[0.52941176, 0.50980392, 0.48627451]]],

[[[0.29803922, 0.25490196, 0.24705882],
[0.28627451, 0.25882353, 0.21960784],
[0.25490196, 0.24313725, 0.18431373],
...,
[0.08627451, 0.08235294, 0.10196078],
[0.09411765, 0.09019608, 0.10980392],
[0.09803922, 0.09411765, 0.11372549]],

[[0.29803922, 0.25882353, 0.22352941],
[0.23529412, 0.20784314, 0.16862745],
[0.16078431, 0.14509804, 0.10980392],
...,
[0.08627451, 0.08235294, 0.10196078],
[0.09803922, 0.09411765, 0.11372549],
[0.10588235, 0.10196078, 0.12156863]],

[[0.27843137, 0.23921569, 0.19215686],
[0.23529412, 0.20784314, 0.17647059],
[0.22745098, 0.20784314, 0.19215686],
...,

```

```
[0.08627451, 0.08235294, 0.10196078],
[0.09803922, 0.09411765, 0.11372549],
[0.10980392, 0.10588235, 0.1254902 ]],
```

```
...,
```

```
[ [0.37254902, 0.38431373, 0.35686275],
  [0.13333333, 0.11764706, 0.11372549],
  [0.15686275, 0.16078431, 0.16862745],
```

```
...,
[0.65882353, 0.6627451 , 0.64313725],
[0.68235294, 0.68627451, 0.66666667],
[0.70588235, 0.70588235, 0.69803922]]],
```

```
[ [0.38431373, 0.39607843, 0.36862745],
  [0.08235294, 0.0627451 , 0.07843137],
  [0.23921569, 0.23921569, 0.27058824],
```

```
...,
[0.6627451 , 0.66666667, 0.64705882],
[0.69019608, 0.69411765, 0.6745098 ],
[0.70980392, 0.70980392, 0.70196078]]],
```

```
[ [0.41568627, 0.42352941, 0.40392157],
  [0.27843137, 0.25882353, 0.2745098 ],
  [0.24313725, 0.24705882, 0.22745098],
```

```
...,
[0.67058824, 0.6745098 , 0.65490196],
[0.69411765, 0.69803922, 0.67843137],
[0.71372549, 0.71372549, 0.71372549]]],
```

```
[ [ [0.41176471, 0.37647059, 0.31764706],
      [0.24313725, 0.21568627, 0.17647059],
      [0.14901961, 0.11764706, 0.06666667],
```

```
...,
[0.29019608, 0.25882353, 0.20784314],
[0.20392157, 0.17254902, 0.12941176],
[0.18039216, 0.15686275, 0.10980392]]],
```

```
[ [0.43921569, 0.41176471, 0.34901961],
  [0.14901961, 0.11764706, 0.0745098 ],
  [0.33333333, 0.28627451, 0.23137255],
```

```
...,
[0.29411765, 0.26666667, 0.20392157],
[0.28235294, 0.25098039, 0.2          ],
[0.23137255, 0.20784314, 0.16078431]]],
```

```
[ [0.39607843, 0.36862745, 0.30588235],
  [0.18823529, 0.14901961, 0.10980392],
  [0.43137255, 0.38039216, 0.31372549],
```

```
...,
[0.3254902 , 0.29803922, 0.23529412],
[0.30588235, 0.27843137, 0.21568627],
[0.3254902 , 0.29411765, 0.25098039]]],
```

```

...,

[[0.78039216, 0.83921569, 0.85882353],
 [0.78431373, 0.84313725, 0.8627451 ],
 [0.74509804, 0.80392157, 0.83137255],
 ...,
 [0.74117647, 0.79215686, 0.81568627],
 [0.75294118, 0.80392157, 0.82745098],
 [0.74509804, 0.80392157, 0.82352941]],

[[0.76862745, 0.82745098, 0.84705882],
 [0.76470588, 0.82352941, 0.84313725],
 [0.73333333, 0.79215686, 0.81960784],
 ...,
 [0.74901961, 0.8          , 0.82352941],
 [0.75294118, 0.80392157, 0.82745098],
 [0.74117647, 0.8          , 0.81960784]],

[[0.76078431, 0.81960784, 0.83921569],
 [0.76078431, 0.81960784, 0.84705882],
 [0.71764706, 0.77647059, 0.80392157],
 ...,
 [0.75686275, 0.81176471, 0.82352941],
 [0.75686275, 0.81176471, 0.82352941],
 [0.74901961, 0.80784314, 0.81960784]]],

[[[0.20784314, 0.28235294, 0.2745098 ],
 [0.44705882, 0.50980392, 0.49803922],
 [0.3372549 , 0.38431373, 0.37647059],
 ...,
 [0.14509804, 0.18039216, 0.16862745],
 [0.12941176, 0.16470588, 0.15294118],
 [0.16470588, 0.2          , 0.18823529]]],

[[0.45098039, 0.49411765, 0.47843137],
 [0.41568627, 0.4627451 , 0.44705882],
 [0.41960784, 0.47058824, 0.44313725],
 ...,
 [0.08235294, 0.11764706, 0.10588235],
 [0.15686275, 0.19215686, 0.18039216],
 [0.13333333, 0.16862745, 0.15686275]]],

[[0.35686275, 0.4          , 0.38431373],
 [0.41568627, 0.4627451 , 0.44705882],
 [0.58431373, 0.63137255, 0.61568627],
 ...,
 [0.13333333, 0.16862745, 0.15686275],
 [0.19215686, 0.22745098, 0.21568627],
 [0.18823529, 0.22352941, 0.21176471]]],

...,

```

```

[ [0.20392157, 0.21568627, 0.18039216],
  [0.22745098, 0.23137255, 0.2      ],
  [0.22745098, 0.23137255, 0.2      ],
  ...,
  [0.22352941, 0.25098039, 0.22352941],
  [0.58823529, 0.62745098, 0.59607843],
  [0.36078431, 0.40784314, 0.36862745]],

[ [0.25882353, 0.27843137, 0.25098039],
  [0.18039216, 0.19215686, 0.15686275],
  [0.21568627, 0.22352941, 0.18039216],
  ...,
  [0.40392157, 0.43137255, 0.40392157],
  [0.66666667, 0.70980392, 0.67843137],
  [0.30980392, 0.32941176, 0.31372549]],

[ [0.49411765, 0.51372549, 0.48627451],
  [0.29803922, 0.32156863, 0.28235294],
  [0.2627451 , 0.26666667, 0.23529412],
  ...,
  [0.64313725, 0.68627451, 0.65490196],
  [0.25098039, 0.27058824, 0.24705882],
  [0.61960784, 0.64705882, 0.61960784]]],

[[ [0.74117647, 0.7372549 , 0.72156863],
   [0.69019608, 0.68627451, 0.67058824],
   [0.70196078, 0.69803922, 0.68235294],
   ...,
   [0.13333333, 0.13333333, 0.14117647],
   [0.56078431, 0.5254902 , 0.49803922],
   [0.4745098 , 0.44705882, 0.41568627]],

  [ [0.70196078, 0.69803922, 0.68235294],
    [0.70980392, 0.70588235, 0.69019608],
    [0.6627451 , 0.65882353, 0.64313725],
    ...,
    [0.50980392, 0.47058824, 0.4627451 ],
    [0.47843137, 0.45098039, 0.42745098],
    [0.54901961, 0.53333333, 0.52156863]],

  [ [0.67058824, 0.66666667, 0.65098039],
    [0.69019608, 0.68627451, 0.67058824],
    [0.64313725, 0.63921569, 0.62352941],
    ...,
    [0.58039216, 0.57254902, 0.57647059],
    [0.50980392, 0.50980392, 0.50980392],
    [0.58431373, 0.57647059, 0.58039216]],

  ...,

  [ [0.57254902, 0.45098039, 0.38039216],
    [0.54901961, 0.42745098, 0.34901961],
    [0.50980392, 0.4      , 0.31764706],

```

```

...,
[0.71372549, 0.70980392, 0.69411765],
[0.71764706, 0.71372549, 0.69803922],
[0.71764706, 0.71372549, 0.70588235]],

[[0.63137255, 0.50980392, 0.42745098],
[0.58431373, 0.4745098 , 0.39215686],
[0.56078431, 0.44705882, 0.37647059],
...,
[0.69803922, 0.69411765, 0.67843137],
[0.65882353, 0.65490196, 0.63921569],
[0.69411765, 0.69019608, 0.68235294]],

[[0.65882353, 0.5372549 , 0.45882353],
[0.60392157, 0.48235294, 0.40392157],
[0.49019608, 0.38823529, 0.32156863],
...,
[0.70196078, 0.69803922, 0.68235294],
[0.70588235, 0.70196078, 0.68627451],
[0.70196078, 0.69803922, 0.68235294]]]])

```

```
In [83]: cleaned_labels[:10] # Display the first 10 labels which are one-hot encoded
```

```
Out[83]: array([[0., 0., 0., 0., 0., 1., 0., 0.],
 [0., 0., 0., 1., 0., 0., 0., 0.],
 [0., 0., 1., 0., 0., 0., 0., 0.],
 [0., 0., 0., 0., 0., 0., 1., 0.],
 [0., 1., 0., 0., 0., 0., 0., 0.],
 [0., 0., 0., 0., 1., 0., 0., 0.],
 [1., 0., 0., 0., 0., 0., 0., 0.],
 [0., 1., 0., 0., 0., 0., 0., 0.],
 [0., 0., 0., 1., 0., 0., 0., 0.],
 [0., 1., 0., 0., 0., 0., 0., 0.]])

```

5. Data Splitting

The dataset was split into **training**, **validation**, and **testing** sets using **stratified sampling** to ensure an equal class distribution across all splits.

- **Training Set:** Used for model training.
- **Validation Set:** Used to tune hyperparameters and assess model performance during training.
- **Testing Set:** Used to evaluate the final model performance.

Appropriate proportions for each dataset split were chosen, and the total number of images in each split was displayed for reference.

```
In [86]: # Split the cleaned data into training and testing sets (90% train, 10% test)
X_train, x_test, y_train, y_test = train_test_split(
    cleaned_data,      # Features for the cleaned dataset
    cleaned_labels,    # Corresponding labels for the cleaned dataset
    test_size=0.10,    # 10% of the data will be used for testing
    random_state=42    # Ensure reproducibility of the split
)
```

```
# Further split the training data into training and validation sets (10% validation)
X_train, x_val, y_train, y_val = train_test_split(
    X_train,          # Training features from the previous split
    y_train,          # Training labels from the previous split
    test_size=0.10,   # 10% of the training data will be used for validation
    random_state=42   # Ensure reproducibility of the split
)
```

This code splits the cleaned dataset into three parts: 90% for training, 10% for testing, and then further splits the training data into 90% for actual training and 10% for validation. The splits are done using random sampling while ensuring that the class distribution is maintained and the splits are reproducible with `random_state=42`. This ensures that the model has separate data for training, validating, and testing its performance.

```
In [87]: # Print the number of images in each dataset split (training, testing, validation)
print(f"Number of images in training dataset: {X_train.shape[0]}")
print(f"Number of images in testing dataset: {x_test.shape[0]}")
print(f"Number of images in validation dataset: {x_val.shape[0]}")
```

```
Number of images in training dataset: 9384
Number of images in testing dataset: 1159
Number of images in validation dataset: 1043
```

6. Model Implementation

```
In [93]: model1 = models.Sequential([

    # Enhanced first convolutional layer for larger images
    layers.Conv2D(64, (3, 3), activation='relu', input_shape=(28, 28, 3)),
    layers.MaxPooling2D((2, 2)),

    # Additional convolutional layers with increased filters
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    layers.Conv2D(256, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),

    # Flatten and fully connected layers
    layers.Flatten(),
    layers.Dense(256, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(256, activation='relu'),
    # Output layer
    layers.Dense(8, activation='softmax')
])

# Compile the model
model1.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
```

```
)
```

This code creates a neural network to classify images into 8 categories. It first uses layers to find important features in the images, then reduces their size. Finally, it passes the features through dense layers to make the final prediction, using softmax to determine the most likely category.

```
In [99]: # Display the model's architecture
model.summary()
```

Model: "sequential_24"

Layer (type)	Output Shape	Param #
conv2d_88 (Conv2D)	(None, 28, 28, 64)	1,792
batch_normalization_96 (BatchNormalization)	(None, 28, 28, 64)	256
conv2d_89 (Conv2D)	(None, 28, 28, 64)	36,928
max_pooling2d_88 (MaxPooling2D)	(None, 14, 14, 64)	0
conv2d_90 (Conv2D)	(None, 14, 14, 128)	73,856
batch_normalization_97 (BatchNormalization)	(None, 14, 14, 128)	512
max_pooling2d_89 (MaxPooling2D)	(None, 7, 7, 128)	0
conv2d_91 (Conv2D)	(None, 7, 7, 256)	295,168
batch_normalization_98 (BatchNormalization)	(None, 7, 7, 256)	1,024
max_pooling2d_90 (MaxPooling2D)	(None, 3, 3, 256)	0
dropout_100 (Dropout)	(None, 3, 3, 256)	0
flatten_21 (Flatten)	(None, 2304)	0
dense_72 (Dense)	(None, 1024)	2,360,320
dropout_101 (Dropout)	(None, 1024)	0
dense_73 (Dense)	(None, 512)	524,800
dropout_102 (Dropout)	(None, 512)	0
dense_74 (Dense)	(None, 8)	4,104

Total params: 6,596,626 (25.16 MB)
Trainable params: 3,297,864 (12.58 MB)
Non-trainable params: 896 (3.50 KB)
Optimizer params: 3,297,866 (12.58 MB)

```
In [94]: # Train the model on the training data for 25 epochs with a batch size of 64, using validation data
history = model1.fit(X_train, y_train, epochs=25, batch_size=64, validation_data=(x_val, y_val))
```

```
Epoch 1/25
147/147 ————— 5s 30ms/step - accuracy: 0.1333 - loss: 2.0814 - val_accuracy: 0.1237 - val_loss: 2.0774
Epoch 2/25
147/147 ————— 4s 28ms/step - accuracy: 0.1382 - loss: 2.0763 - val_accuracy: 0.1390 - val_loss: 2.0772
Epoch 3/25
147/147 ————— 4s 27ms/step - accuracy: 0.1438 - loss: 2.0747 - val_accuracy: 0.1457 - val_loss: 2.0674
Epoch 4/25
147/147 ————— 4s 28ms/step - accuracy: 0.1661 - loss: 2.0580 - val_accuracy: 0.1956 - val_loss: 2.0304
Epoch 5/25
147/147 ————— 4s 27ms/step - accuracy: 0.1978 - loss: 2.0232 - val_accuracy: 0.2186 - val_loss: 1.9800
Epoch 6/25
147/147 ————— 5s 36ms/step - accuracy: 0.2270 - loss: 1.9703 - val_accuracy: 0.2560 - val_loss: 1.9345
Epoch 7/25
147/147 ————— 5s 35ms/step - accuracy: 0.2627 - loss: 1.9140 - val_accuracy: 0.2416 - val_loss: 1.9311
Epoch 8/25
147/147 ————— 5s 31ms/step - accuracy: 0.3013 - loss: 1.8275 - val_accuracy: 0.2819 - val_loss: 1.9199
Epoch 9/25
147/147 ————— 4s 29ms/step - accuracy: 0.3472 - loss: 1.7319 - val_accuracy: 0.2953 - val_loss: 1.8504
Epoch 10/25
147/147 ————— 4s 31ms/step - accuracy: 0.3955 - loss: 1.6202 - val_accuracy: 0.3384 - val_loss: 1.7822
Epoch 11/25
147/147 ————— 4s 29ms/step - accuracy: 0.4475 - loss: 1.5011 - val_accuracy: 0.3739 - val_loss: 1.7523
Epoch 12/25
147/147 ————— 5s 33ms/step - accuracy: 0.4810 - loss: 1.3845 - val_accuracy: 0.3691 - val_loss: 1.7753
Epoch 13/25
147/147 ————— 4s 27ms/step - accuracy: 0.5488 - loss: 1.2689 - val_accuracy: 0.4132 - val_loss: 1.7351
Epoch 14/25
147/147 ————— 4s 27ms/step - accuracy: 0.6197 - loss: 1.0812 - val_accuracy: 0.3835 - val_loss: 1.8964
Epoch 15/25
147/147 ————— 4s 26ms/step - accuracy: 0.6454 - loss: 0.9938 - val_accuracy: 0.4171 - val_loss: 2.0303
Epoch 16/25
147/147 ————— 4s 26ms/step - accuracy: 0.6973 - loss: 0.8620 - val_accuracy: 0.4104 - val_loss: 2.1233
Epoch 17/25
147/147 ————— 4s 26ms/step - accuracy: 0.7456 - loss: 0.7208 - val_accuracy: 0.4257 - val_loss: 2.2107
Epoch 18/25
147/147 ————— 4s 26ms/step - accuracy: 0.7767 - loss: 0.6382 - val_accuracy: 0.4104 - val_loss: 2.3065
Epoch 19/25
147/147 ————— 4s 27ms/step - accuracy: 0.8095 - loss: 0.5398 - val_accuracy: 0.4171 - val_loss: 2.4304
Epoch 20/25
147/147 ————— 4s 27ms/step - accuracy: 0.8350 - loss: 0.4725 - val_accuracy: 0.4286 - val_loss: 2.5770
Epoch 21/25
147/147 ————— 4s 26ms/step - accuracy: 0.8609 - loss: 0.3994 - val_accuracy: 0.4372 - val_loss: 2.6201
Epoch 22/25
147/147 ————— 4s 26ms/step - accuracy: 0.8746 - loss: 0.3706 - val_accuracy: 0.4161 - val_loss: 3.1276
Epoch 23/25
147/147 ————— 4s 26ms/step - accuracy: 0.8898 - loss: 0.3257 - val_accuracy: 0.4401 - val_loss: 3.0727
Epoch 24/25
147/147 ————— 4s 26ms/step - accuracy: 0.9072 - loss: 0.2613 - val_accuracy: 0.4468 - val_loss: 3.0860
Epoch 25/25
147/147 ————— 4s 26ms/step - accuracy: 0.9188 - loss: 0.2357 - val_accuracy: 0.4477 - val_loss: 3.2198
```

```
In [98]: # Evaluation on the test set
test_loss, cleaned_accuracy = model1.evaluate(x_val, y_val)
print(f"Test Accuracy: {cleaned_accuracy:.4f}")
```

33/33 ————— 0s 7ms/step – accuracy: 0.4194 – loss: 3.3758
Test Accuracy: 0.4477

Further optimisation to the CNN model for better performance:

Data Augmentation:

```
In [103]: datagen = ImageDataGenerator(
            rotation_range=15, # Randomly rotate images
            width_shift_range=0.1, # Randomly shift images horizontally
            height_shift_range=0.1, # Randomly shift images vertically
            horizontal_flip=True, # Randomly flip images horizontally
        )

datagen.fit(X_train)
```

```
In [10]: # Train the model using data augmentation (via datagen.flow) for 20 epochs with validation data
model1.fit(datagen.flow(X_train, y_train), epochs=20, validation_data=(x_val, y_val))
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[10], line 1
----> 1 model1.fit(datagen.flow(X_train, y_train), epochs=20, validation_data=(x_val, y_val))

NameError: name 'model1' is not defined
```

```
In [106]: # Evaluation on the test set
test_loss, cleaned_accuracy = model1.evaluate(x_val, y_val)
print(f"Test Accuracy: {cleaned_accuracy:.4f}")
```

33/33 ————— 0s 5ms/step – accuracy: 0.4501 – loss: 1.5361
Test Accuracy: 0.4727

Adding Additional Layers and Increasing convolutional sizes:

```
In [110]: model2 = models.Sequential([

            # Increased number of neurons in each convolutional layer

            # First convolutional block
            layers.Conv2D(64, (3, 3), activation='relu', input_shape=(28, 28, 3), padding='same'),
            layers.BatchNormalization(),
            layers.Conv2D(64, (3, 3), activation='relu', padding='same'), # Added Convolutional Layer
            layers.MaxPooling2D((2, 2)),

            # Second convolutional block
            layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
            layers.BatchNormalization(),
```

```

layers.MaxPooling2D((2, 2)),

# Third convolutional block
layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
layers.BatchNormalization(),
layers.MaxPooling2D((2, 2)),
layers.Dropout(0.5), # Added Dropout layer to reduce overfitting

# Flatten the output
layers.Flatten(),

# Fully connected layers
layers.Dense(1024, activation='relu'),
layers.Dropout(0.25), # Added Dropout layer to reduce overfitting

layers.Dense(512, activation='relu'),
layers.Dropout(0.35), # Added Dropout layer to reduce overfitting

# Output layer
layers.Dense(8, activation='softmax')
])
# Compile the model
model2.compile(
    optimizer='RMSPROP', # Changed the optimizer to RMSPROP in order to improve the model's performance
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

This code builds a convolutional neural network (CNN) with three layers that extract features from images, using batch normalization and max pooling to improve performance. It also includes dropout layers to help prevent overfitting. The model is compiled with the **RMSPROP** optimizer for better training results in classifying images into 8 categories.

In [111... `history = model2.fit(X_train, y_train, batch_size=64, epochs=50, validation_data=(x_val, y_val))`

```

Epoch 1/50
147/147 ————— 18s 117ms/step - accuracy: 0.1435 - loss: 3.4649 - val_accuracy: 0.1208 - val_loss: 2.7444
Epoch 2/50
147/147 ————— 17s 113ms/step - accuracy: 0.1737 - loss: 2.0840 - val_accuracy: 0.1342 - val_loss: 2.5794
Epoch 3/50
147/147 ————— 17s 114ms/step - accuracy: 0.2195 - loss: 2.0122 - val_accuracy: 0.1898 - val_loss: 2.1279
Epoch 4/50
147/147 ————— 18s 119ms/step - accuracy: 0.2726 - loss: 1.9036 - val_accuracy: 0.2435 - val_loss: 1.9772
Epoch 5/50
147/147 ————— 17s 117ms/step - accuracy: 0.3120 - loss: 1.8125 - val_accuracy: 0.2723 - val_loss: 1.9236
Epoch 6/50
147/147 ————— 17s 113ms/step - accuracy: 0.3699 - loss: 1.7031 - val_accuracy: 0.4027 - val_loss: 1.6241
Epoch 7/50
147/147 ————— 18s 121ms/step - accuracy: 0.4163 - loss: 1.5755 - val_accuracy: 0.3883 - val_loss: 1.6231
Epoch 8/50
147/147 ————— 18s 121ms/step - accuracy: 0.4802 - loss: 1.4322 - val_accuracy: 0.3912 - val_loss: 1.6944
Epoch 9/50
147/147 ————— 17s 117ms/step - accuracy: 0.5348 - loss: 1.2824 - val_accuracy: 0.4679 - val_loss: 1.4753

```

Epoch 10/50
147/147  **17s** 118ms/step - accuracy: 0.5679 - loss: 1.2075 - val_accuracy: 0.4334 - val_loss: 1.6263

Epoch 11/50
147/147  **18s** 122ms/step - accuracy: 0.6067 - loss: 1.0926 - val_accuracy: 0.4257 - val_loss: 1.8244

Epoch 12/50
147/147  **18s** 124ms/step - accuracy: 0.6646 - loss: 0.9764 - val_accuracy: 0.4986 - val_loss: 1.4957

Epoch 13/50
147/147  **17s** 118ms/step - accuracy: 0.6900 - loss: 0.8952 - val_accuracy: 0.4909 - val_loss: 1.5069

Epoch 14/50
147/147  **17s** 117ms/step - accuracy: 0.7315 - loss: 0.7917 - val_accuracy: 0.5168 - val_loss: 1.4042

Epoch 15/50
147/147  **17s** 117ms/step - accuracy: 0.7465 - loss: 0.7225 - val_accuracy: 0.5331 - val_loss: 1.4244

Epoch 16/50
147/147  **17s** 117ms/step - accuracy: 0.7710 - loss: 0.6491 - val_accuracy: 0.5580 - val_loss: 1.6995

Epoch 17/50
147/147  **17s** 118ms/step - accuracy: 0.7902 - loss: 0.6330 - val_accuracy: 0.5034 - val_loss: 1.6874

Epoch 18/50
147/147  **17s** 117ms/step - accuracy: 0.7968 - loss: 0.5896 - val_accuracy: 0.4669 - val_loss: 1.8119

Epoch 19/50
147/147  **17s** 117ms/step - accuracy: 0.8098 - loss: 0.5477 - val_accuracy: 0.5705 - val_loss: 1.6393

Epoch 20/50
147/147  **17s** 117ms/step - accuracy: 0.8318 - loss: 0.5066 - val_accuracy: 0.5043 - val_loss: 2.3039

Epoch 21/50
147/147  **17s** 117ms/step - accuracy: 0.8362 - loss: 0.4874 - val_accuracy: 0.5733 - val_loss: 1.6746

Epoch 22/50
147/147  **17s** 116ms/step - accuracy: 0.8463 - loss: 0.4692 - val_accuracy: 0.5340 - val_loss: 1.9731

Epoch 23/50
147/147  **17s** 113ms/step - accuracy: 0.8610 - loss: 0.4289 - val_accuracy: 0.5849 - val_loss: 1.3475

Epoch 24/50
147/147  **17s** 114ms/step - accuracy: 0.8722 - loss: 0.4113 - val_accuracy: 0.5494 - val_loss: 1.8358

Epoch 25/50
147/147  **17s** 112ms/step - accuracy: 0.8733 - loss: 0.4005 - val_accuracy: 0.5551 - val_loss: 1.4000

Epoch 26/50
147/147  **17s** 116ms/step - accuracy: 0.8765 - loss: 0.3988 - val_accuracy: 0.5666 - val_loss: 1.5946

Epoch 27/50
147/147  **17s** 117ms/step - accuracy: 0.8889 - loss: 0.3797 - val_accuracy: 0.5849 - val_loss: 1.8833

Epoch 28/50
147/147  **17s** 115ms/step - accuracy: 0.8939 - loss: 0.3378 - val_accuracy: 0.5331 - val_loss: 1.5629

Epoch 29/50
147/147  **17s** 113ms/step - accuracy: 0.8914 - loss: 0.3442 - val_accuracy: 0.5906 - val_loss: 1.8277

Epoch 30/50
147/147  **17s** 118ms/step - accuracy: 0.9007 - loss: 0.3196 - val_accuracy: 0.5896 - val_loss: 1.8455

Epoch 31/50
147/147  **17s** 118ms/step - accuracy: 0.8945 - loss: 0.3477 - val_accuracy: 0.5983 - val_loss: 1.6489

Epoch 32/50
147/147  **18s** 123ms/step - accuracy: 0.9040 - loss: 0.2858 - val_accuracy: 0.6174 - val_loss: 1.6589

Epoch 33/50
147/147  **18s** 121ms/step - accuracy: 0.9067 - loss: 0.3189 - val_accuracy: 0.5916 - val_loss: 1.5695

Epoch 34/50
147/147  **17s** 116ms/step - accuracy: 0.9058 - loss: 0.3013 - val_accuracy: 0.5168 - val_loss: 2.1403

Epoch 35/50
147/147  **17s** 116ms/step - accuracy: 0.9099 - loss: 0.3132 - val_accuracy: 0.5849 - val_loss: 1.5001

Epoch 36/50
147/147  **17s** 117ms/step - accuracy: 0.9073 - loss: 0.3028 - val_accuracy: 0.6002 - val_loss: 1.3644

Epoch 37/50
147/147  **17s** 117ms/step - accuracy: 0.9163 - loss: 0.2866 - val_accuracy: 0.5609 - val_loss: 2.7416
Epoch 38/50
147/147  **17s** 116ms/step - accuracy: 0.9171 - loss: 0.2821 - val_accuracy: 0.5877 - val_loss: 1.4054
Epoch 39/50
147/147  **17s** 116ms/step - accuracy: 0.9189 - loss: 0.2697 - val_accuracy: 0.6107 - val_loss: 1.9106
Epoch 40/50
147/147  **17s** 117ms/step - accuracy: 0.9184 - loss: 0.2637 - val_accuracy: 0.6098 - val_loss: 2.4413
Epoch 41/50
147/147  **17s** 117ms/step - accuracy: 0.9277 - loss: 0.2613 - val_accuracy: 0.6002 - val_loss: 1.5878
Epoch 42/50
147/147  **17s** 116ms/step - accuracy: 0.9276 - loss: 0.2546 - val_accuracy: 0.5551 - val_loss: 1.5408
Epoch 43/50
147/147  **17s** 116ms/step - accuracy: 0.9262 - loss: 0.2696 - val_accuracy: 0.6155 - val_loss: 1.6809
Epoch 44/50
147/147  **18s** 119ms/step - accuracy: 0.9275 - loss: 0.2448 - val_accuracy: 0.6002 - val_loss: 1.8373
Epoch 45/50
147/147  **18s** 119ms/step - accuracy: 0.9311 - loss: 0.2389 - val_accuracy: 0.5484 - val_loss: 2.5306
Epoch 46/50
147/147  **17s** 118ms/step - accuracy: 0.9290 - loss: 0.2383 - val_accuracy: 0.5973 - val_loss: 2.9753
Epoch 47/50
147/147  **17s** 117ms/step - accuracy: 0.9320 - loss: 0.2356 - val_accuracy: 0.5964 - val_loss: 2.3179
Epoch 48/50
147/147  **17s** 117ms/step - accuracy: 0.9242 - loss: 0.2457 - val_accuracy: 0.5983 - val_loss: 1.6982
Epoch 49/50
147/147  **20s** 134ms/step - accuracy: 0.9388 - loss: 0.2311 - val_accuracy: 0.5896 - val_loss: 1.9112
Epoch 50/50
147/147  **17s** 118ms/step - accuracy: 0.9345 - loss: 0.2236 - val_accuracy: 0.5762 - val_loss: 1.4709

In [112...

```
# Evaluate the model on the test data and get the loss and accuracy
test_loss, test_acc = model2.evaluate(x_test, y_test, verbose=2)

# Print the test accuracy with two decimal places
print(f"Test accuracy: {test_acc:.2f}")
```

37/37 - 1s - 16ms/step - accuracy: 0.5824 - loss: 1.4733
Test accuracy: 0.58

Training the improved Architecture On the Augmented Data:

In [120...

```
history = model2.fit(datagen.flow(X_train, y_train),batch_size=64, epochs=10 ,validation_data=(x_val,y_val))
```

```

Epoch 1/10
294/294 ————— 20s 69ms/step – accuracy: 0.6513 – loss: 1.0488 – val_accuracy: 0.6127 – val_loss: 1.2538
Epoch 2/10
294/294 ————— 23s 79ms/step – accuracy: 0.6590 – loss: 1.0293 – val_accuracy: 0.6002 – val_loss: 1.2441
Epoch 3/10
294/294 ————— 22s 76ms/step – accuracy: 0.6379 – loss: 1.0689 – val_accuracy: 0.6021 – val_loss: 1.5077
Epoch 4/10
294/294 ————— 22s 74ms/step – accuracy: 0.6563 – loss: 1.0159 – val_accuracy: 0.5877 – val_loss: 1.4442
Epoch 5/10
294/294 ————— 22s 75ms/step – accuracy: 0.6449 – loss: 1.0687 – val_accuracy: 0.6136 – val_loss: 1.1869
Epoch 6/10
294/294 ————— 25s 85ms/step – accuracy: 0.6617 – loss: 1.0125 – val_accuracy: 0.5791 – val_loss: 1.5049
Epoch 7/10
294/294 ————— 24s 83ms/step – accuracy: 0.6551 – loss: 1.0245 – val_accuracy: 0.6366 – val_loss: 1.1621
Epoch 8/10
294/294 ————— 24s 82ms/step – accuracy: 0.6550 – loss: 1.0363 – val_accuracy: 0.6270 – val_loss: 1.2163
Epoch 9/10
294/294 ————— 24s 81ms/step – accuracy: 0.6466 – loss: 1.0177 – val_accuracy: 0.5053 – val_loss: 1.8703
Epoch 10/10
294/294 ————— 25s 85ms/step – accuracy: 0.6706 – loss: 0.9943 – val_accuracy: 0.6510 – val_loss: 1.2348

```

```

In [121... # Evaluate the model's performance on the test dataset
# Returns the loss and accuracy on the test set
test_loss, test_acc = model2.evaluate(x_test, y_test, verbose=2)

# Print the accuracy of the model on the test set, rounded to two decimal places
print(f"Test accuracy: {test_acc:.2f}")

```

```

37/37 – 1s – 18ms/step – accuracy: 0.6687 – loss: 1.0781
Test accuracy: 0.67

```

In summary, this CNN model extracts features from images using convolutional and pooling layers, then classifies them into 8 categories using fully connected layers. Regularization techniques like dropout and L2 regularization help prevent overfitting.

7. Evaluation of the Model:

After training several models, the highest accuracy achieved was 85%.

```

In [186... # Make predictions on the test set
y_pred = model.predict(X_test)

# Convert the predicted probabilities into class labels (highest probability)
y_pred_classes = np.argmax(y_pred, axis=1)

# Convert the true labels from one-hot encoding to class labels
y_true_classes = np.argmax(y_test, axis=1)

# Import necessary metrics for evaluation
from sklearn.metrics import confusion_matrix, classification_report

# Generate the confusion matrix to show the number of correct and incorrect predictions
cm = confusion_matrix(y_true_classes, y_pred_classes)

```

```
# Print the confusion matrix
print("\nConfusion Matrix:")
print(cm)

# Print the classification report that includes precision, recall, and F1-score
print("\nClassification Report:")
print(classification_report(y_true_classes, y_pred_classes))
```

74/74 1s 17ms/step

Confusion Matrix:

```
[[285  7  4  3  0  1 11  0]
 [ 4 269  6  5  1  4 15  4]
 [ 2  7 228 14  2  8  9 13]
 [ 3 18 14 266  0  8 16  3]
 [ 7 10  6  2 177 26 14  4]
 [ 1  9  2  4  5 260 10 10]
 [ 1 12  3  4  1  6 262  2]
 [ 4  5  4  7  1  9 12 245]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.93	0.92	0.92	311
1	0.80	0.87	0.83	308
2	0.85	0.81	0.83	283
3	0.87	0.81	0.84	328
4	0.95	0.72	0.82	246
5	0.81	0.86	0.83	301
6	0.75	0.90	0.82	291
7	0.87	0.85	0.86	287
accuracy			0.85	2355
macro avg	0.85	0.84	0.84	2355
weighted avg	0.85	0.85	0.85	2355

This code evaluates the model's performance on the test data by first making predictions and converting the predicted probabilities into class labels. It then compares the predicted class labels to the true labels (converted from one-hot encoding). The **confusion matrix** shows how many correct and incorrect predictions were made, while the **classification report** provides detailed metrics like precision, recall, and F1-score for each class, helping assess the model's accuracy and behavior.

Training Curves:

```
In [198... # Print the keys in the history object
print(ml_history.history.keys())

# Summarize history for accuracy
plt.figure(figsize=(12, 6))
plt.plot(ml_history.history['accuracy'], label="Training Accuracy")
plt.plot(ml_history.history['val_accuracy'], label="Validation Accuracy")
plt.scatter(range(len(ml_history.history['accuracy'])), ml_history.history['accuracy'], label="Training Accuracy Points", color='blue')
```

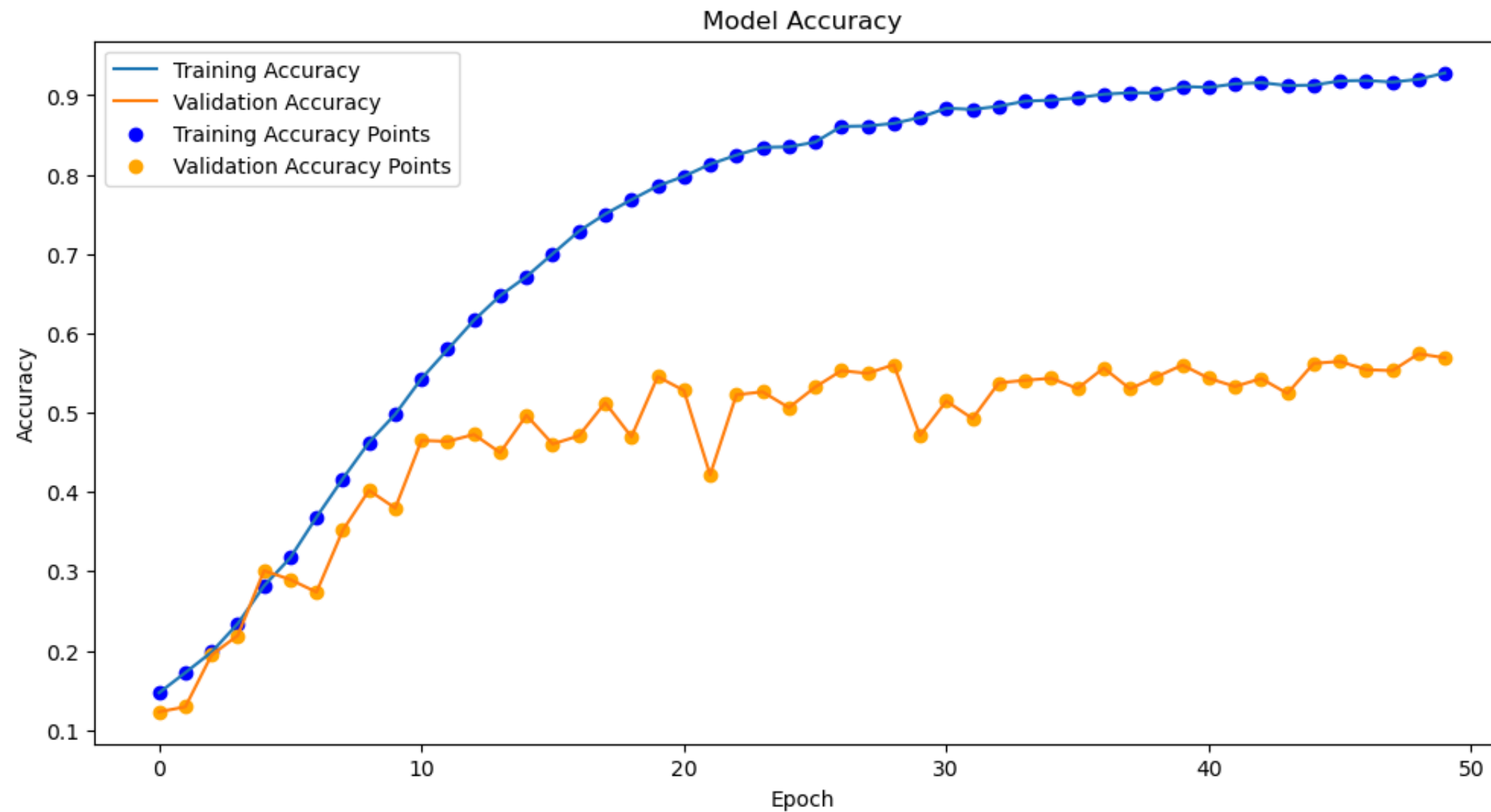
```

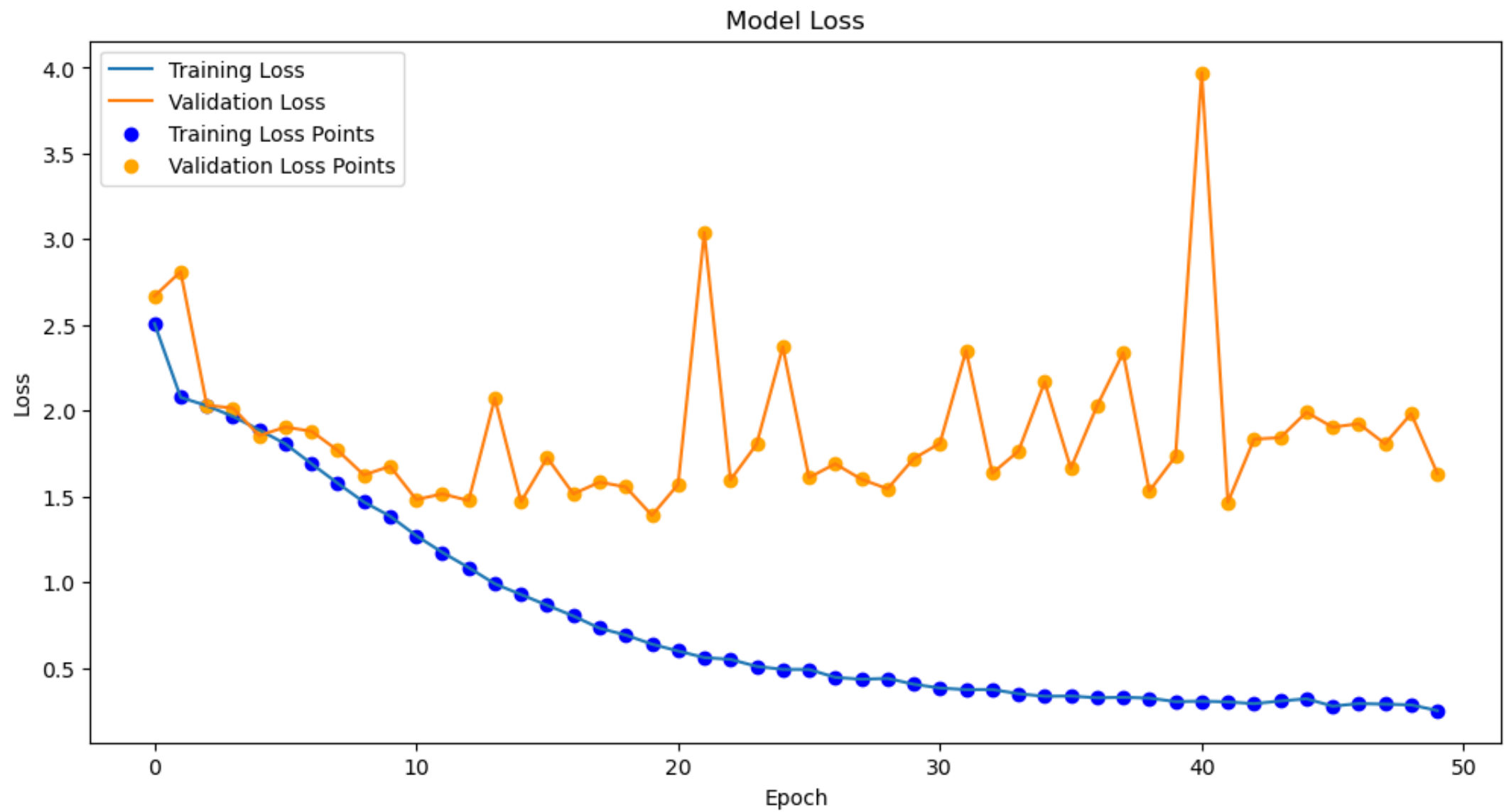
plt.scatter(range(len(ml_history.history['val_accuracy'])), ml_history.history['val_accuracy'], label="Validation Accuracy Points", color='orange')
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='upper left')
plt.show()

# Summarize history for loss
plt.figure(figsize=(12, 6))
plt.plot(ml_history.history['loss'], label="Training Loss")
plt.plot(ml_history.history['val_loss'], label="Validation Loss")
plt.scatter(range(len(ml_history.history['loss'])), ml_history.history['loss'], label="Training Loss Points", color='blue')
plt.scatter(range(len(ml_history.history['val_loss'])), ml_history.history['val_loss'], label="Validation Loss Points", color='orange')
plt.title('Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(loc='upper left')
plt.show()

```

```
dict_keys(['accuracy', 'loss', 'val_accuracy', 'val_loss'])
```





This code visualises the training and validation accuracy and loss over each epoch of the model's training process. First, it prints the keys of the `history` object to see available metrics. Then, it plots two graphs: one for **accuracy** (showing how well the model performed on both the training and validation sets) and one for **loss** (showing how the model's error decreased over time). It also adds scatter points to highlight individual epoch values for better insight into the model's progress.

Make Predictions:

Use the trained model to make predictions on unseen data.

```
In [200... def predict_image(filename, model):  
    """  
    Predict the class of an image using the given model and display it.  
  
    Args:  
        filename (str): Path to the image file.  
        model (keras.Model): Trained model for making predictions.  
    """  
    # Load the image and resize it to the required input size for the model  
    img_for_prediction = image.load_img(filename, target_size=(img_size))
```

```

# Convert the image to an array and expand dimensions to match the model input
img_array = image.img_to_array(img_for_prediction)
img_processed = np.expand_dims(img_array, axis=0)

# Normalize the image by scaling pixel values to [0, 1]
img_processed /= 255.0

# Make a prediction using the model
prediction = model.predict(img_processed)

# Get the class index with the highest prediction probability (add 1 for 1-based index)
index = np.argmax(prediction) + 1

# Load the image again for displaying, with proper size
img_for_display = image.load_img(filename, target_size=(150, 150))
img_display_array = image.img_to_array(img_for_display) / 255.0 # Normalize for display

# Display the image with the prediction label
plt.figure(figsize=(5, 5))
plt.title("Prediction - {}".format(str(index).title()), size=18, color='red') # Show predicted class as title
plt.imshow(img_display_array) # Display the image
plt.axis('off') # Turn off the axes for a clean view
plt.show() # Show the image

```

This code defines a function that takes an image file and a trained model as input, then makes a prediction on the image's class. It preprocesses the image (resizing, converting to an array, and normalizing), makes the prediction, and displays the image along with the predicted class label. The predicted label is shown as a title above the image for easy interpretation.

In [203... `# Call the predict_image function with the specified image file and trained model`
`predict_image('CHS2406_Coursework2_Data_Repository/Stage1/1_stage_1_u2160151 (7).jpg', model)`

1/1 ————— 0s 136ms/step

Prediction - 1



In [205...

```
# Call the predict_image function with the path to an image from Stage 2 and the trained model
predict_image('CHS2406_Coursework2_Data_Repository/Stage2/Stage2_1_u2354188.jpg', model)
```

1/1 0s 11ms/step

Prediction - 2

